

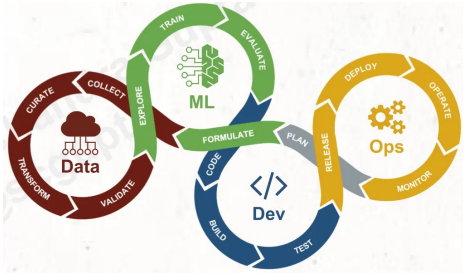
mlflow™

MLOps & MLflow

Prof. Peerapon Vateekul, Ph.D.
Department of Computer Engineering,
Faculty of Engineering, Chulalongkorn University
Peerapon.v@chula.ac.th

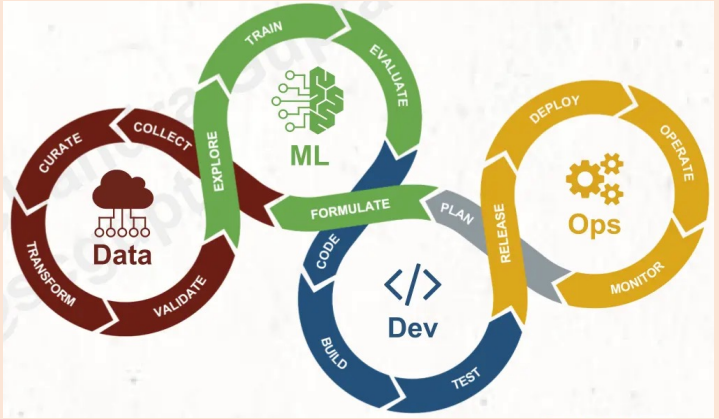
Outline

- MLOps
  - Introduction to MLOps
  - Dev Module Guideline
  - Ops Module Guideline
- MLflow



mlflow™

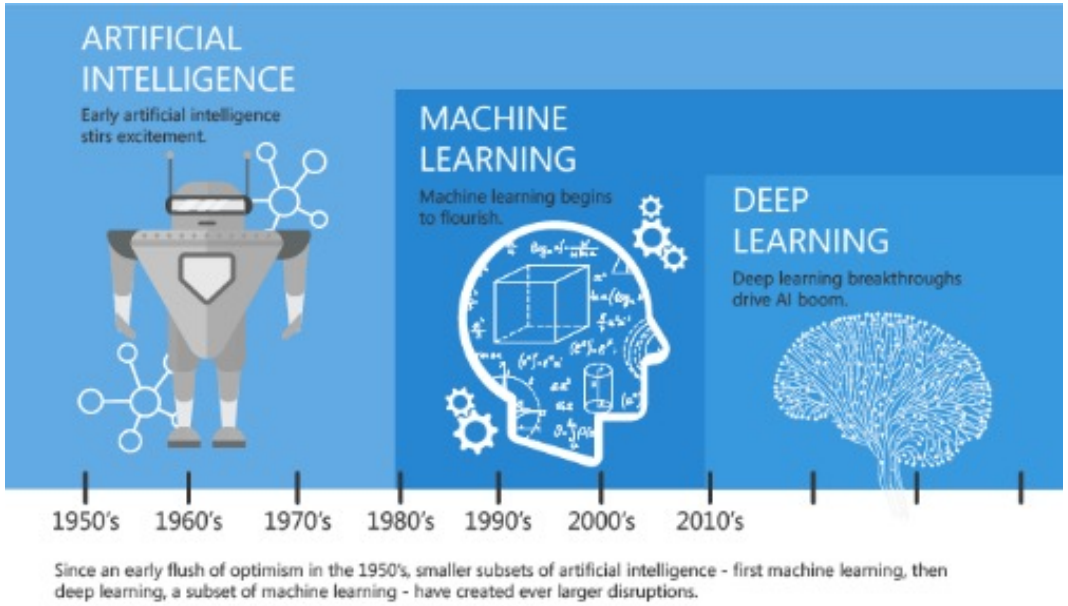
MLOps



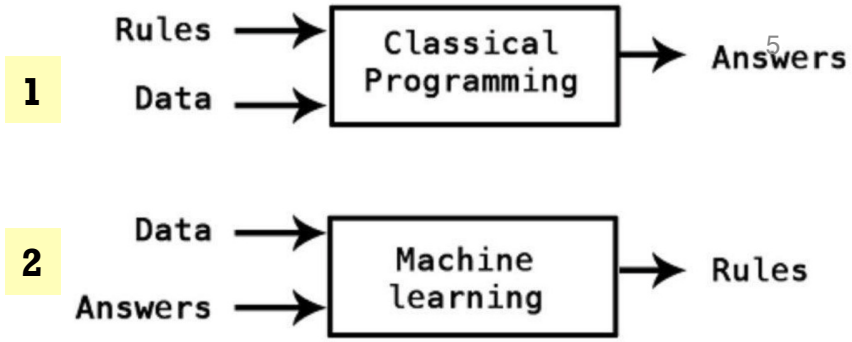
Introduction to MLOps

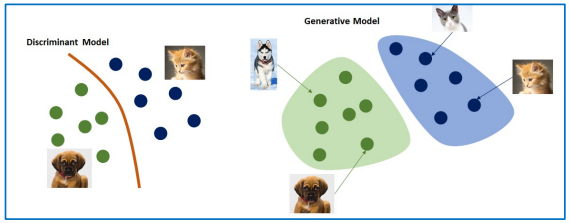
AI = Automation RECAP

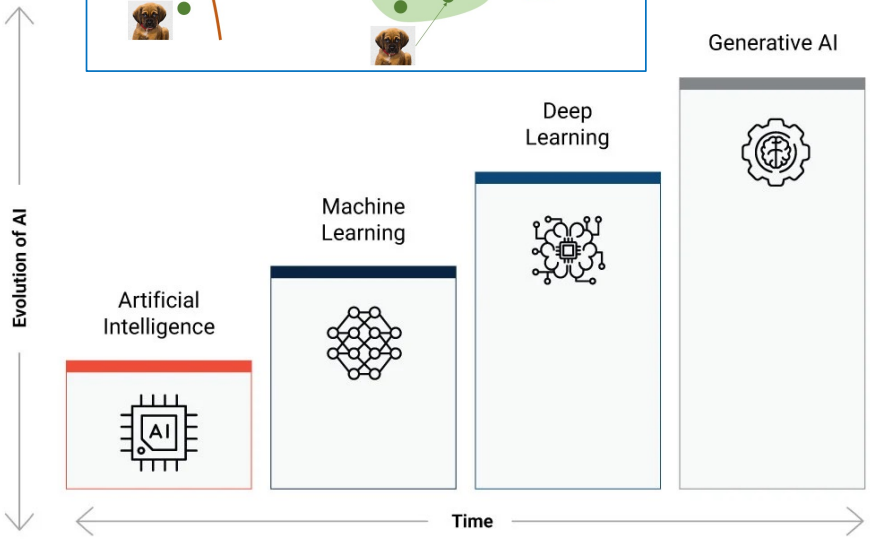
- 1) Rule-based AI
- 2) Machine Learning (ML)



Since an early flush of optimism in the 1950's, smaller subsets of artificial intelligence - first machine learning, then deep learning, a subset of machine learning - have created ever larger disruptions.







https://mc.ai/machine-learning-basics-artificial-intelligence-machine-learning-and-deep-learning/

Do you think that user can use this code to get the prediction result? NO!!!

2_Linear-Regression-v2.ipynb

File Edit View Insert Runtime Tools Help

+ Code + Text Copy to Drive

```
[ ]  1 coeff_df = pd.DataFrame(lm.coef_,lm.feature_names_in_,columns=['Coefficient'])
     2 coeff_df
```

Does this make sense? Probably not because I made up this data. If you want real data to repeat this sort of analysis, check out the boston dataset:

```
from sklearn.datasets import load_boston
boston = load_boston()
print(boston.DESCR)
boston_df = boston.data
```

Predictions from our Model

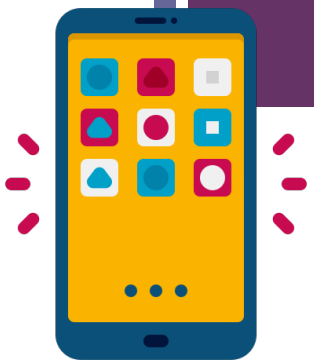
Let's grab predictions off our test set and see how well it did!

```
[ ]  1 predictions = lm.predict(X_test)
```

```
[ ]  1 plt.scatter(y_test,predictions)
```

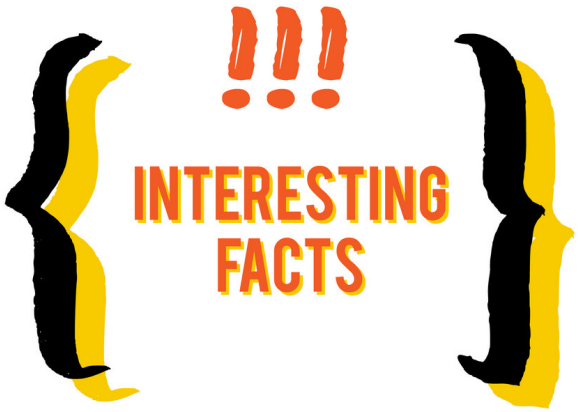
Residual Histogram

```
[ ]  1 sns.distplot((y_test-predictions),bins=50);
```





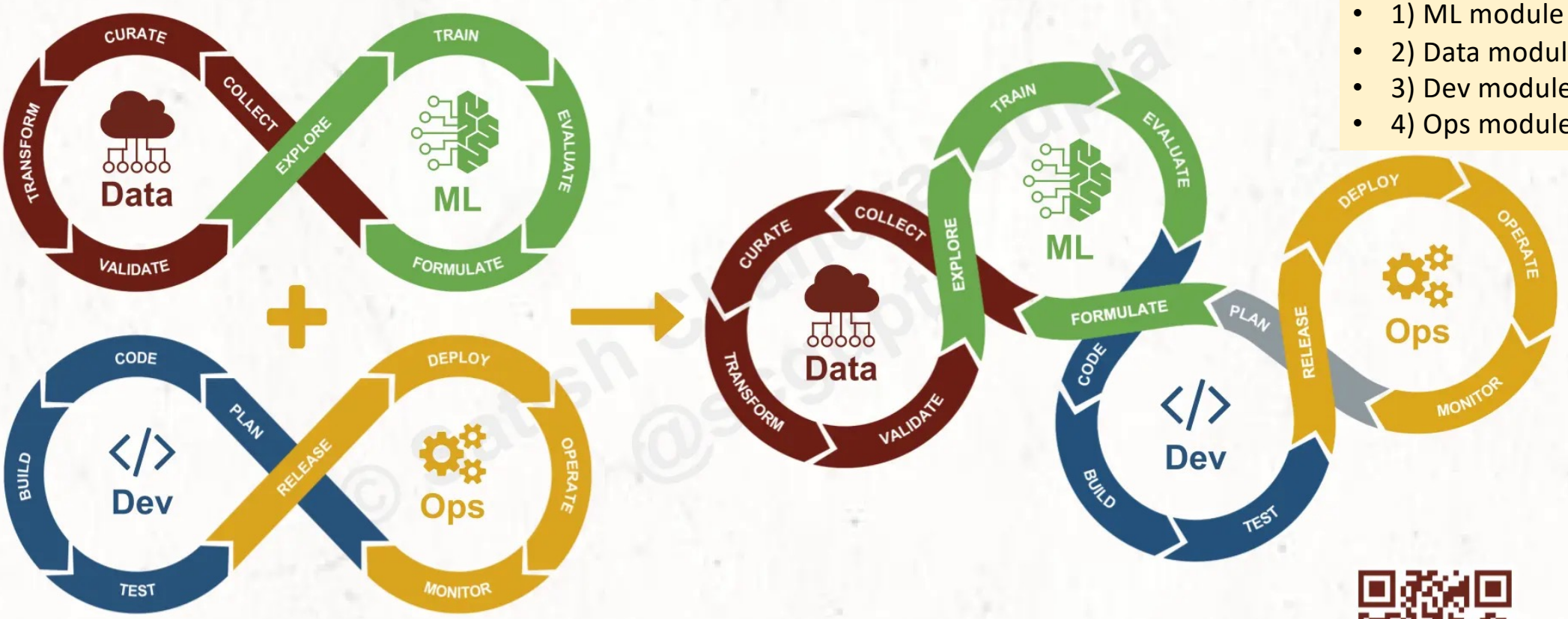
Interesting facts



- More than 50% of AI projects were FAILED since they didn't plan about the deployment.
- For the AI project, finish building the model is only 50% of the work.
- The remaining work is about the deployment as a touchpoint to the target user.

MLOps = DataML + DevOps

ml4devs.com/mlops-lifecycle

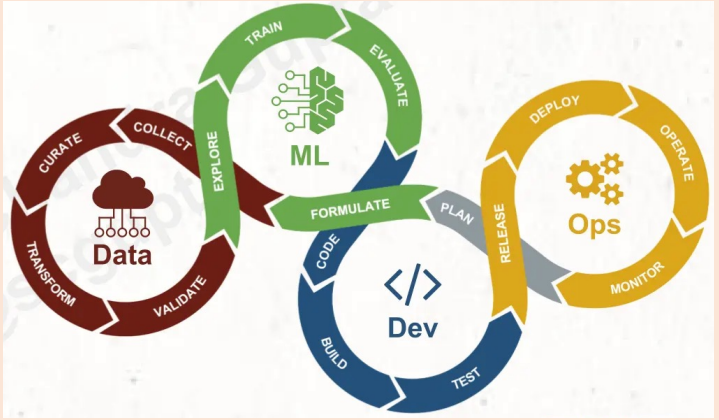


- 1) ML module
- 2) Data module
- 3) Dev module
- 4) Ops module

© 2022 Satish Chandra Gupta
CC BY-NC-ND 4.0 International License
creativecommons.org/licenses/by-nc-nd/4.0/

https://www.ml4devs.com/images/illustrations/ml-lifecycle-fusing-model-and-software-development.webp

scgupta.me
twitter.com/scgupta
linkedin.com/in/scgupta



Dev Module Guideline

Guideline for the dev module



- More than 50% of AI projects were FAILED since they didn't plan about the deployment.

- 1) Understand who is the target user first? E.g., customer vs staff
- 2) Action: Knowledge without action is meaningless. (API & Integration)
- 3) User journey: simple & easy-to-use
- 4) Model is already used via the touchpoint. Then, what next?

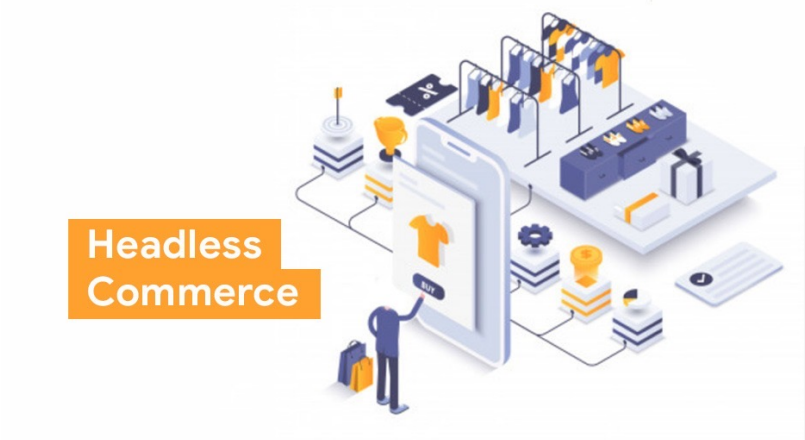
1) Understand who is the target user first?
E.g. customer vs staff

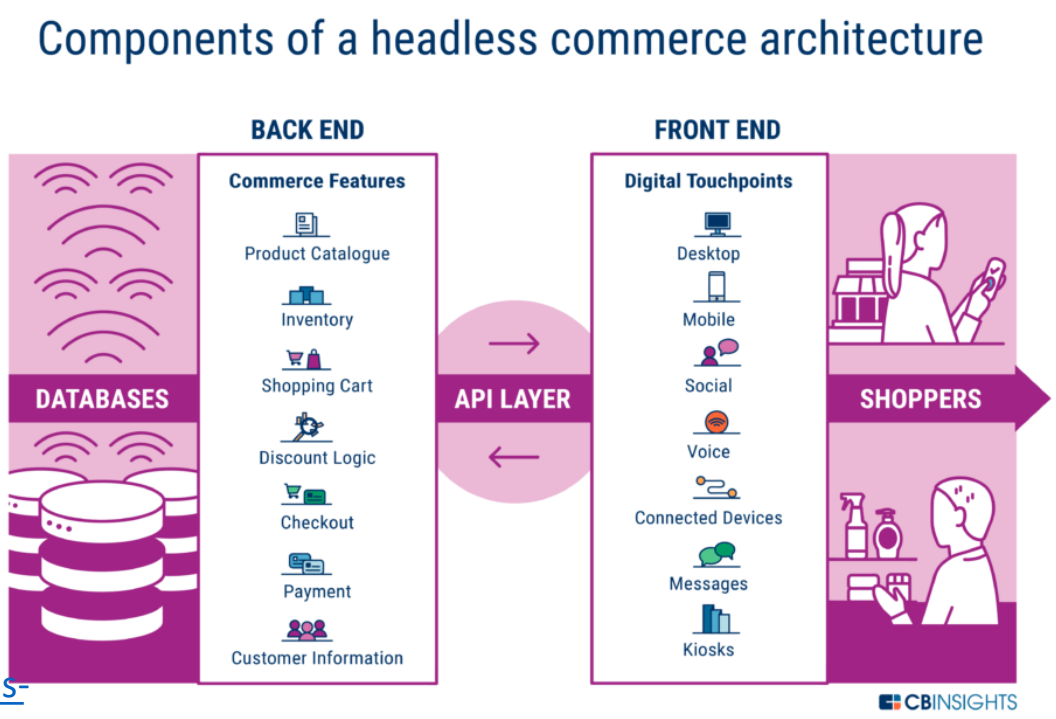




2) Action: Knowledge without action is meaningless. (API & Integration)

- API's for omnichannel
- Campaign automation
- Loyalty management



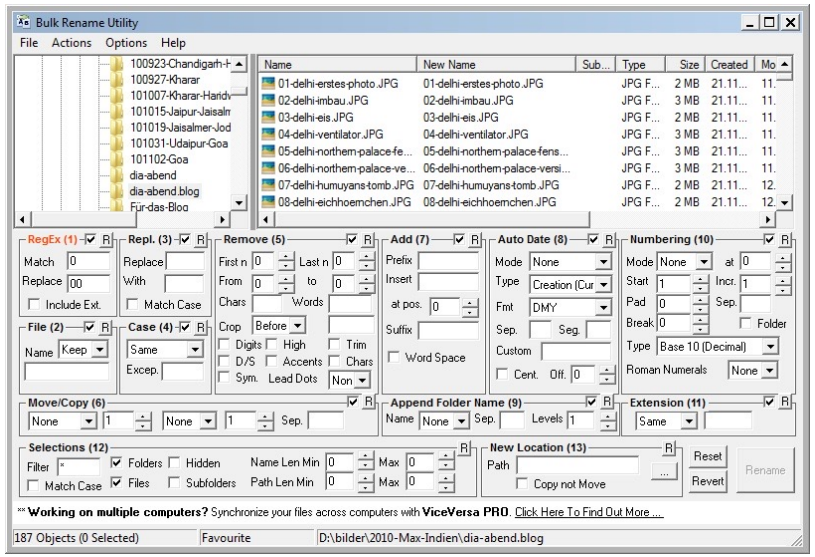


https://www.cbinsights.com/research/report/what-is-headless-commerce/

3) User journey: simple & easy-to-use



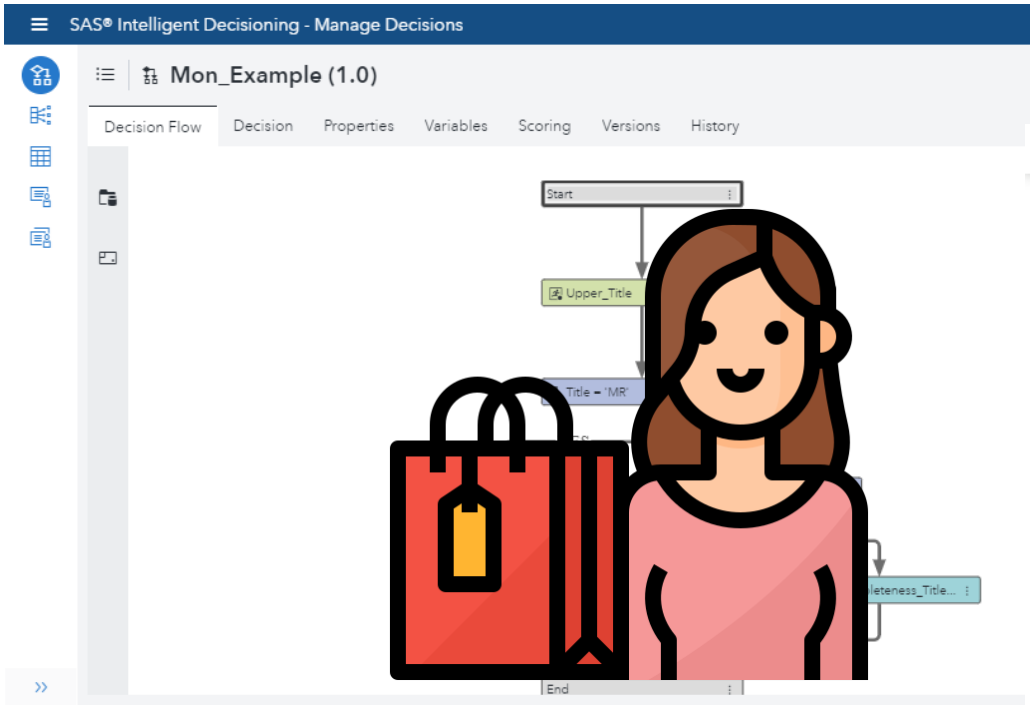


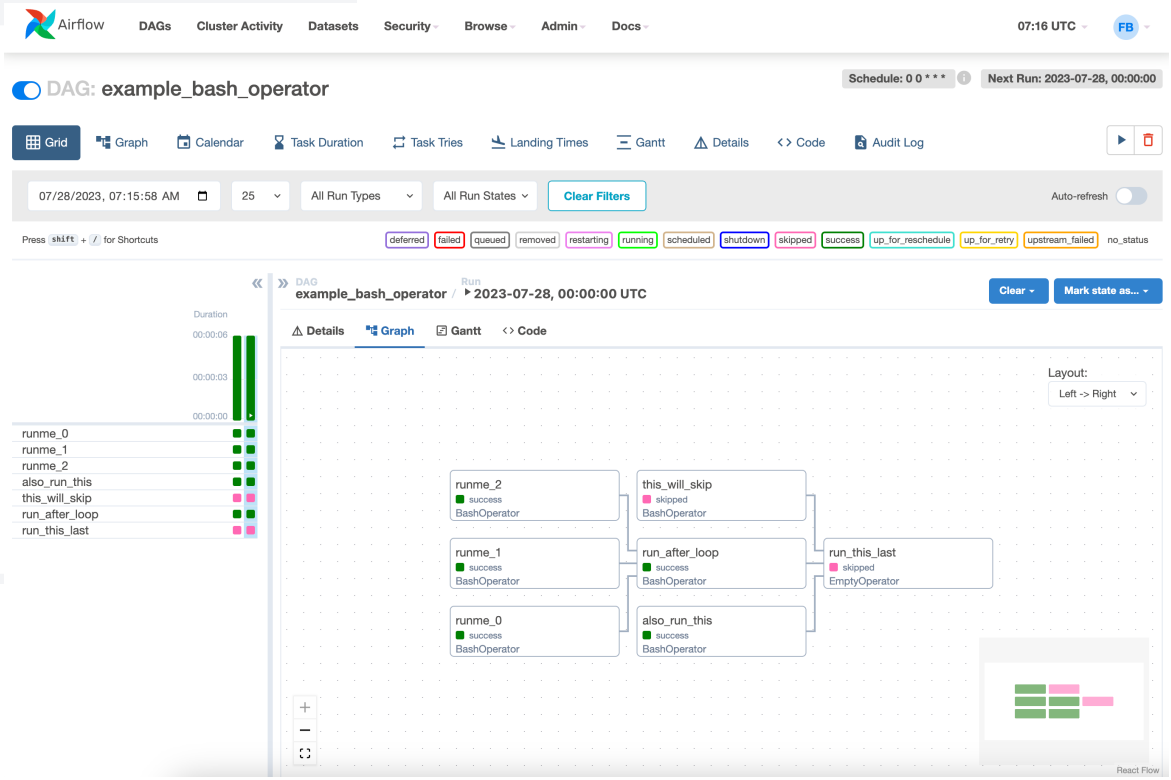


4) Workflow management

Model is already used via the touchpoint. Then, what next?





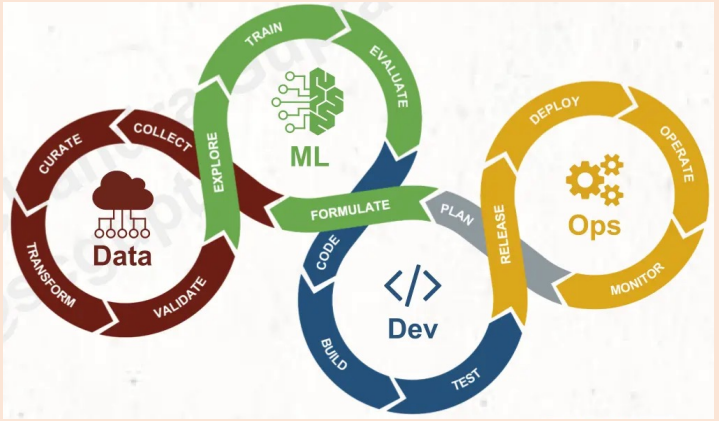


Guideline for the dev module (recap)



- More than 50% of AI projects were FAILED since they didn't plan about the deployment.

- 1) Understand who is the target user first? E.g., customer vs staff
- 2) Action: Knowledge without action is meaningless. (API & Integration)
- 3) User journey: simple & easy-to-use
- 4) Model is already used via the touchpoint. Then, what next?



Ops Module Guideline

Guideline for the ops module



- MLOps (Ops = Operations)
- Release (versioning), configure, monitor

- 1) Monitoring: operation and performance
- 2) Model is not stationary and always needed to be retrained.
- 3) There are many MLOps tools nowadays.
- 4) BI Dashboard can help to summarize the collected data.

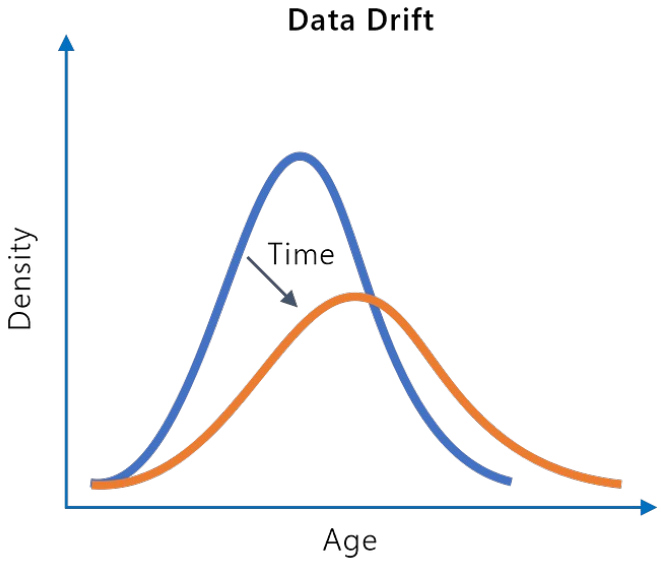
1) Monitoring

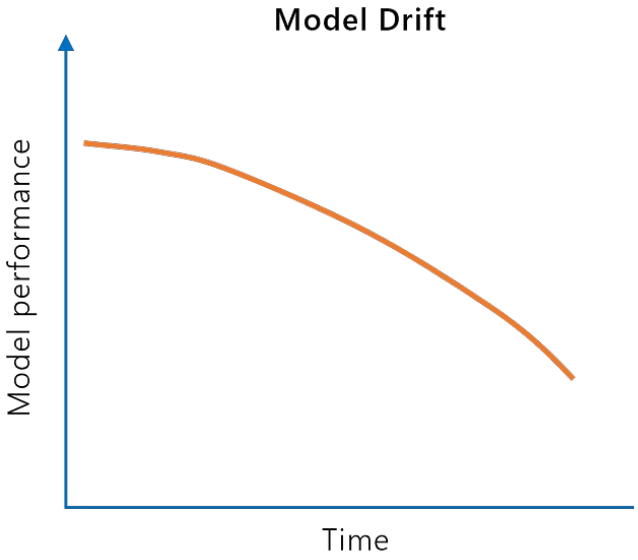


- Operation monitoring



- Performance monitoring





2) Model is not stationary and always needed to be retrained.



3) Many MLOps tools nowadays

Release (versioning), configure, monitor

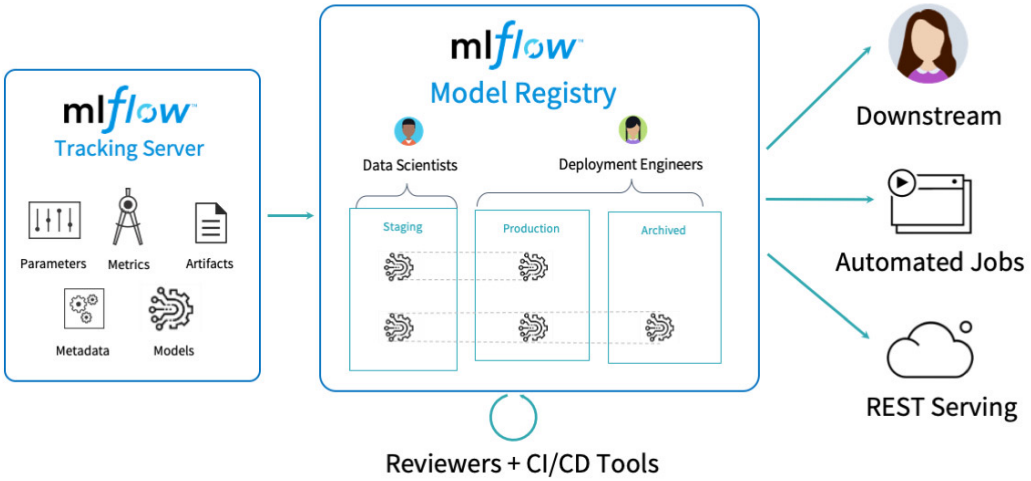


mlflow

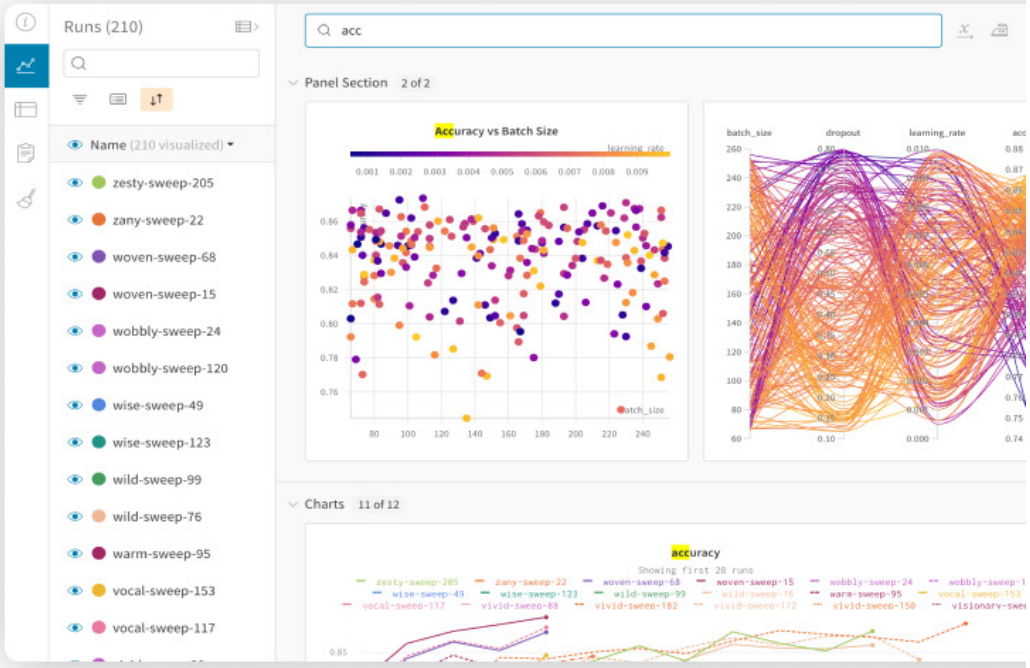
Tracking
Record and query experiments: code, data, config, results

Projects
Packaging format for reproducible runs on any platform

Models
General format for sending models to diverse deploy tools



W&B



4) BI Dashboard can help to summarize the collected data.



Guideline for the ops module (recap)



- MLOps (Ops = Operations)
- Release (versioning), configure, monitor

- 1) Monitoring: operation and performance
- 2) Model is not stationary and always needed to be retrained.
- 3) There are many MLOps tools nowadays.
- 4) BI Dashboard can help to summarize the collected data.

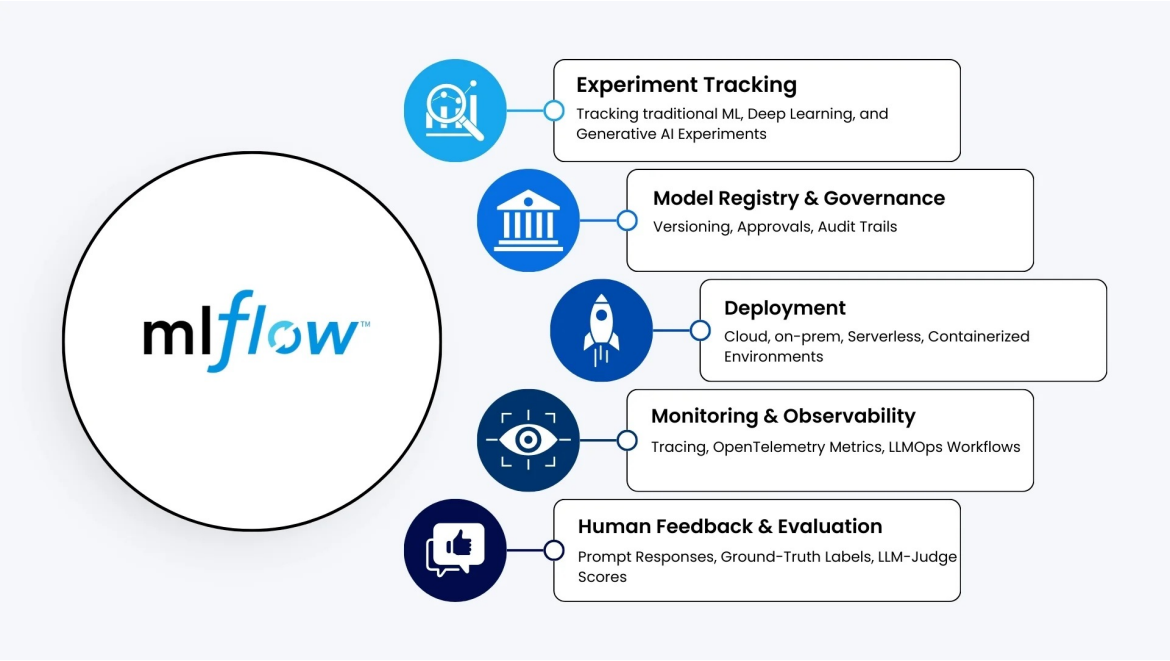
+

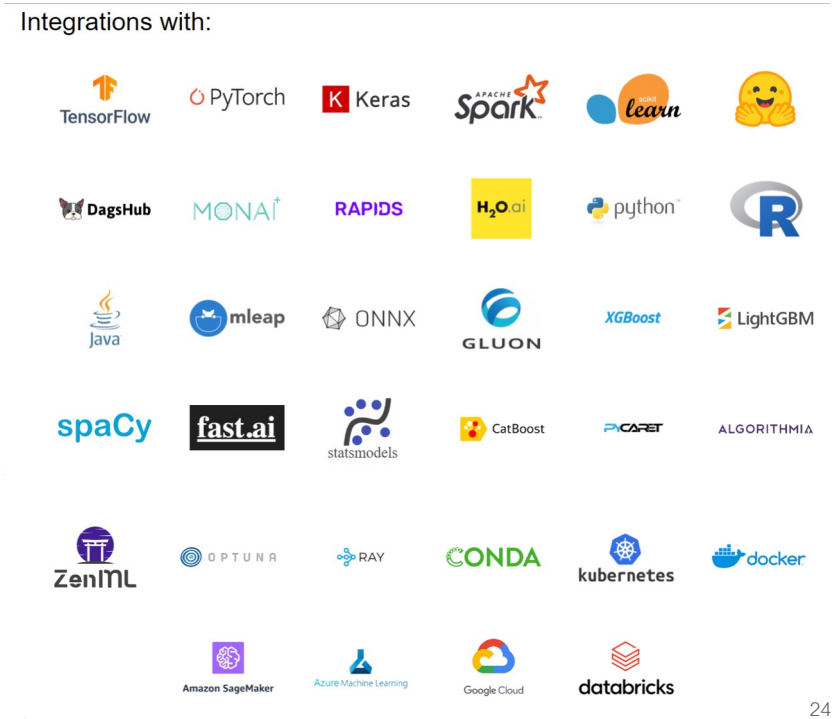
MLflow



What is Mlflow (current version 3.0)?

- MLflow makes it simple to construct end-to-end Machine Learning pipelines in production, and this article will teach you all you need to know about the platform. This implies that at the conclusion of this tutorial, you'll be able to utilize MLflow for Machine Learning pipelines from model experimentation through model deployment.





Overview

How to use MLflow in 4 steps

Example in the notebook: a Decision Tree on the Breast Cancer dataset (455 train / 114 test)



Step 1

Track

Log every training run

mlflow.start_run()
log_param / log_metric / log_model



Step 2

Compare

Find the best run

mlflow.search_runs()





Step 3

Register

Save it as a versioned model

mlflow.register_model()





Step 4

Serve

Expose it as a REST API

mlflow models serve



Setup (once per project)

pip install mlflow · mlflow.set_tracking_uri("sqlite:///mlruns.db") · mlflow.set_experiment("test_experiment")

Step 1 · Track

Wrap your training in a run and log what matters

Each call to train_model() becomes one run in the experiment

```
def train_model(criterion, max_depth):
with mlflow.start_run():
model = DecisionTreeClassifier(criterion=criterion,
max_depth=max_depth, random_state=0)
model.fit(X_train, y_train)
acc = accuracy_score(y_test, model.predict(X_test))

mlflow.log_param("criterion", criterion)
mlflow.log_param("max_depth", max_depth)
mlflow.log_metric("accuracy", acc)

signature = infer_signature(X_train, model.predict(X_train))
mlflow.sklearn.log_model(model, "model", signature=signature)

# Try every combination → 10 separate runs
for criterion, max_depth in itertools.product(["gini","entropy"], [1,2,3,4,5]):

train_model(criterion, max_depth)
```

1. Open a run

Everything inside with mlflow.start_run() belongs to one run

2. Log parameters

The settings you chose (hyperparameters), with log_param

3. Log metrics

Numbers you want to compare later, with log_metric

4. Log the model

log_model saves the trained model; the signature records the input/output shape so it can be served later

See the runs: mlflow ui --backend-store-uri sqlite:///mlruns.db --port 5000 (on Colab, expose it with ngrok)

Steps 2–3 · Compare and Register

Pick the best run and give it a name

From a pile of runs to one model you can refer to as breast_cancer v1

```
# 1) Compare: query runs, best accuracy first
best_run_df = mlflow.search_runs(
order_by=["metrics.accuracy DESC"], max_results=5)

run_id = best_run_df.loc[0, "run_id"]
model_uri = f"runs:/{run_id}/model"

# 2) Register: give the model a name + version
mlflow.register_model(model_uri=model_uri, name="breast_cancer")


# 3) Load by name/version, no run_id needed
loaded = mlflow.pyfunc.load_model(
model_uri="models:/breast_cancer/1")
predicted = loaded.predict(pd.DataFrame(X_test))
```

1. Compare runs

search_runs returns a DataFrame; sort by any logged metric and take row 0

2. Two kinds of model URI

runs:/<run_id>/model points at one run; models:/<name>/<version> points at the Registry

3. Register

The Registry stores models by name with versions 1, 2, 3… so teammates and apps use a stable name, not a random run_id

☆ **Best run in the notebook**

gini · max_depth 2 · 96.49% accuracy

Step 4 · Serve

Turn the registered model into a REST API

mlflow models serve loads models:/breast_cancer/1 and answers HTTP requests

```
# 1) Start a model server (background, port 5001)
MLFLOW_TRACKING_URI=sqlite:///mlruns.db \
mlflow models serve -m "models:/breast_cancer/1" \
-p 5001 --env-manager local

# 2) Wait until GET /ping returns 200
requests.get("http://127.0.0.1:5001/ping")

# 3) Send rows as JSON, get predictions back
payload = {"inputs": X_test.tolist()}
r = requests.post("http://127.0.0.1:5001/invocations",
headers={"Content-Type": "application/json"},
data=json.dumps(payload))
api_pred = r.json()["predictions"] # same as predict()

# 4) Stop it when done
pkill -f "mlflow models serve"
```



1. Serve from the Registry

-m names the model; MLFLOW_TRACKING_URI tells the server where the Registry lives; --env-manager local reuses installed packages

2. Health check

Poll /ping until it answers 200 before sending data

3. Any client works

Input must match the logged signature (30 features per row). curl, JavaScript or a mobile app can call it; no sklearn or MLflow needed on the client

Result: API accuracy 96.49%, exactly equal to in-process predict()

Other MLOps tools

1) Standard MLOps

mlflow™

Weights & Biases

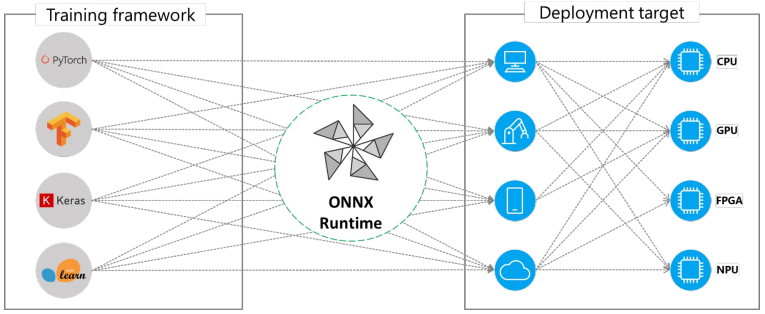
TensorBoard

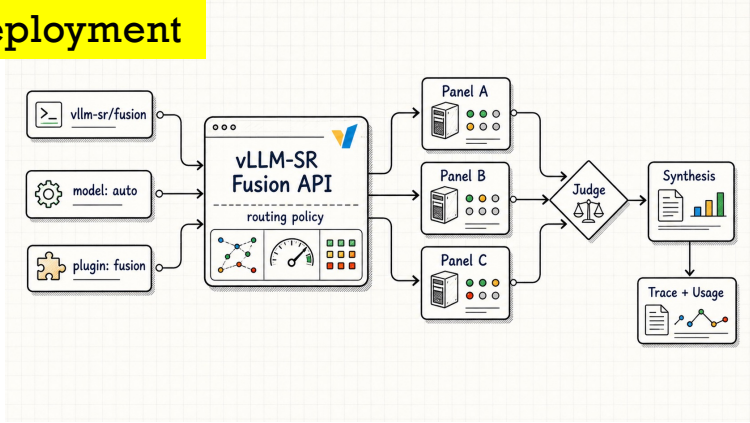
2) LLMOps



3) Model Optimization & Deployment

TensorRT







Peerapon.v@chula.ac.th